



NAVISAFE

Guillem Tarradas i Miquel Calzada
Sistemes Microinformàtics i Xarxes
Institut de Palamós
07/04/2026

INDEX



1. Context i situació inicial
2. Abast i objectius
3. Solució tècnica
4. Planificació del projecte
5. Integració i arquitectura
6. Exemple de web i programa
7. Resultats i validació
8. Conclusions i aprenentatges

CONTEXT I SITUACIÓ INICIAL



Many small boats lack advanced navigation technology.

Difficulties with:

- Maintaining course
- Controlling speed
- Detecting obstacles

High risk in adverse weather conditions.

CONTEXT I SITUACIÓ INICIAL



Navisafe is an easy-to-use assisted navigation system for small boats.

Centralized management with a Raspberry Pi:

- Real-time sensors
- Web dashboard
- Semi-automatic control

Accessible from any device through a browser.



ABAST I OBJECTIUS



Funcionalitats incloses:

- Monitoratge en temps real
- Planificació de rutes amb mapa interactiu
- Navegació per instruments
- Estabilització automàtica
- Detecció d'obstacles
- Dashboard web centralizat
- Gestió de rutes



ABAST I OBJECTIUS



No inclou:

- Hardware propi
- Embarcacions > 15 m d'eslora
- Certificacions oficials
- Integració en vaixells reals



SOLUCIONS TÈCNIQUES



Monitoratge del Vaixell

Capa física (hardware):

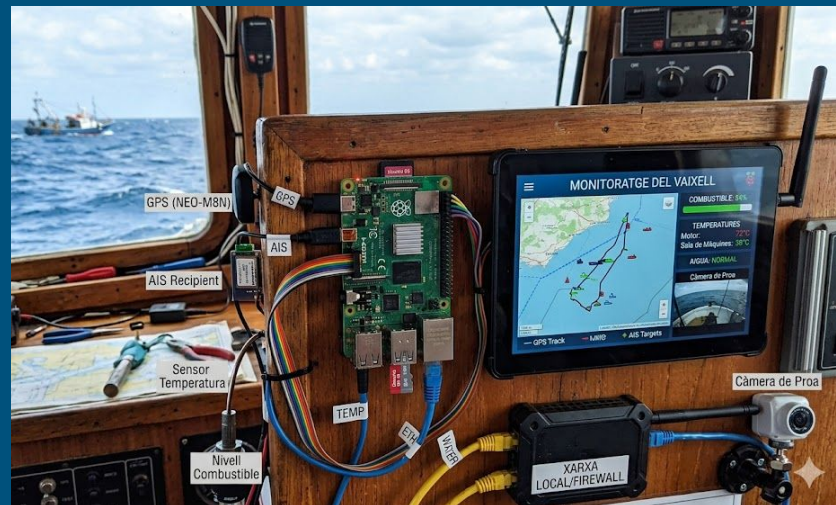
- Sensors: combustible, temperatura, GPS, AIS, càmera i sensor aigua
- Raspberry Pi
- Pantalla tàctil / tablet

Xarxa i comunicacions:

- Wi-Fi / Ethernet local.
- Firewall

Sistema operatiu i entorns virtuals:

- Ubuntu



SOLUCIONS TÈCNIQUES

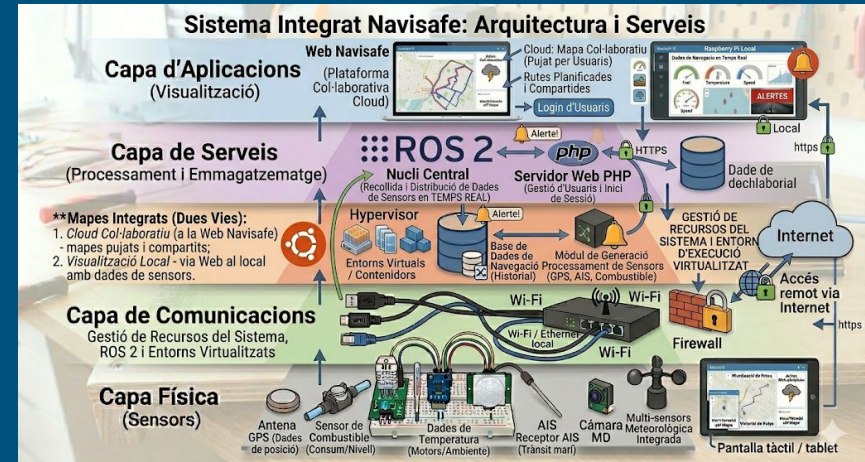


Serveis:

- ROS2 com a nucli central per recollir i distribuir les dades dels sensor que mostra les dades en temps real
- Servidor web PHP per a l'inici de sessió dels usuaris
- Alertes
- https

Aplicacions:

- Web Navisafe
- La raspberry amb visualització de sensors i alertes.
- Mapa 2 maneres. 1r amb cloud que pot ser pujada per la gent, rutes planificades i compartides. El 2n de manera via web



SOLUCIONS TÈCNIQUES



Dades de la navegació

Capa física (hardware):

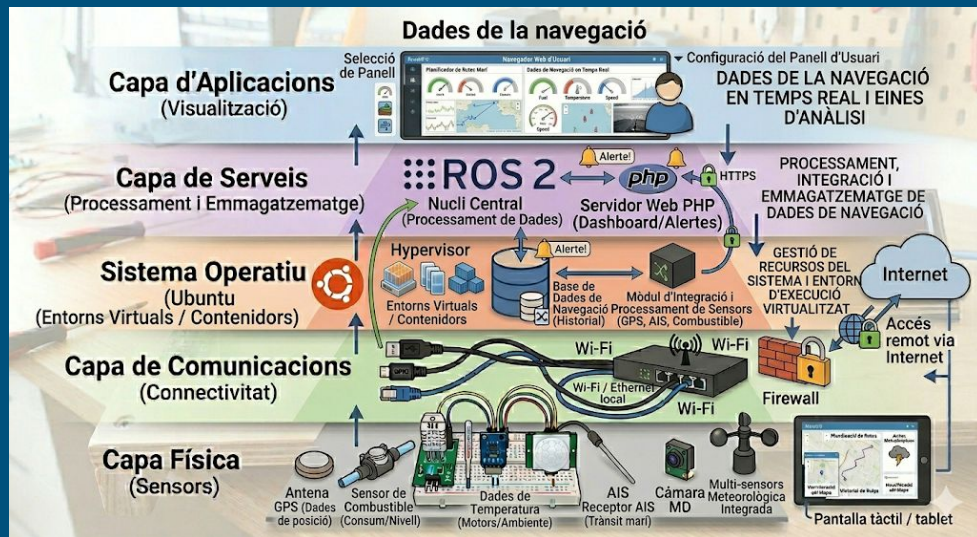
- Sensors: combustible, temperatura, GPS, AIS, càmera
- Raspberry Pi
- Pantalla tàctil / tablet

Xarxa i comunicacions:

- Wi-Fi / Ethernet local.
- Firewall
- Accés remot via Internet
- Comunicació segura

Sistema operatiu i entorns virtuals

- Ubuntu



SOLUCIONS TÈCNIQUES

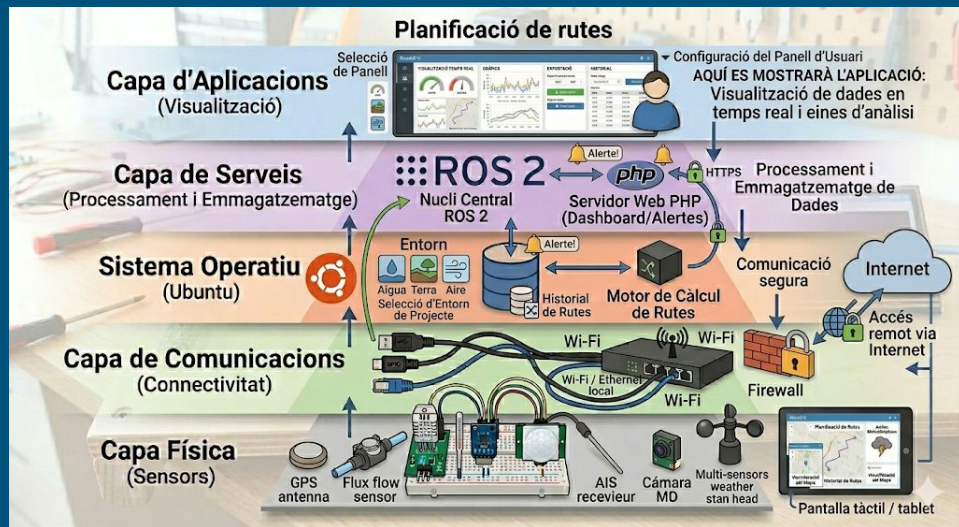


Serveis:

- ROS 2 com a nucli central per recollir, processar i distribuir les dades dels sensors.
- Servidor web PHP que mostra les dades en temps real al dashboard i genera alertes.
- https
- Seleccions el panell que vols i el seu entorn

Aplicacions:

- Visualització de dades en temps real
- Generació de gràfics
- Exportació d'informes
- Historial de dades



SOLUCIONS TÈCNIQUES



Planificació de rutes

Capa física (hardware):

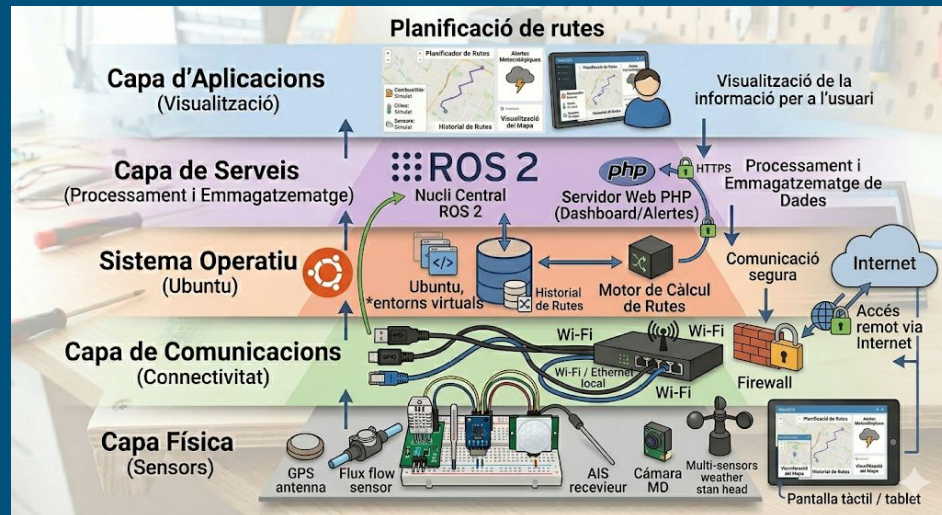
- Sensors simulats: combustible, temperatura, GPS, AIS, càmera i sensors meteorològics
- Raspberry Pi
- Pantalla tàctil / tablet

Xarxa i comunicacions:

- Wi-Fi / Ethernet local
- Firewall
- Accés remot via Internet
- Comunicació segura

Sistema operatiu i entorns virtuals

- Ubuntu



SOLUCIONS TÈCNIQUES

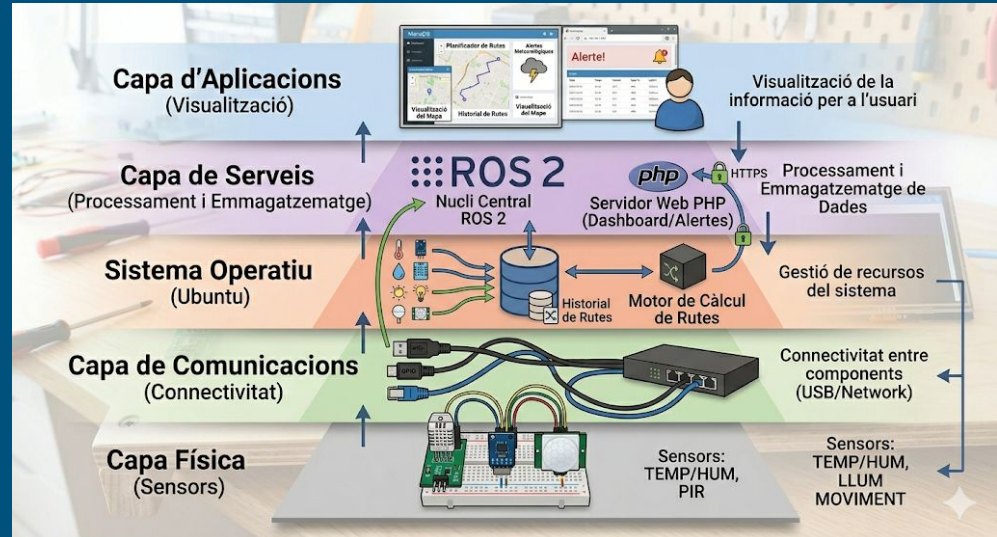


Serveis:

- ROS 2 com a nucli central per recollir, processar i distribuir les dades dels sensors.
- Servidor web PHP que mostra les dades en temps real al dashboard i genera alertes.
- https
- Base de dades per historial de rutes
- Motor de càlcul de rutes

Aplicacions:

- Planificador de rutes
- Visualització del mapa
- Alertes meteorològiques
- Historial de rutes

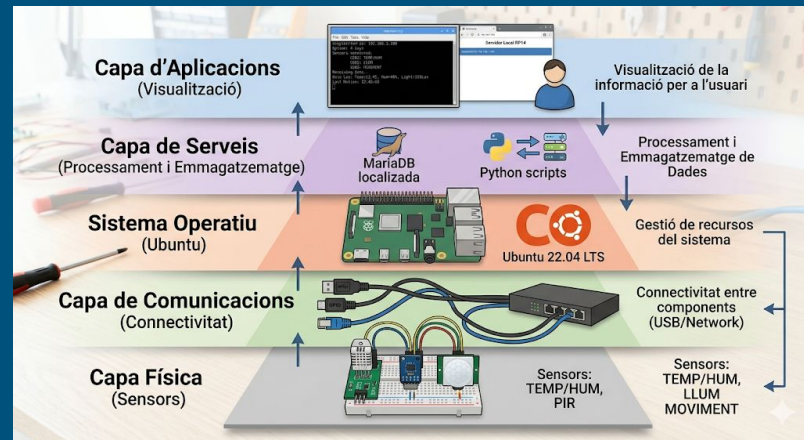


SOLUCIONS TÈCNIQUES



Arquitectura del sistema

- El sistema s'organitza en 5 capes funcionals:
- Capa física: sensors encarregats de captar dades.
- Capa de comunicacions: garanteix la connectivitat entre components.
- Sistema operatiu: Ubuntu, responsable de la gestió de recursos.
- Capa de serveis: processament i emmagatzematge de dades.
- Capa d'aplicacions: visualització de la informació per a l'usuari.



SOLUCIONS TÈCNIQUES



Nucli del sistema

- El nucli és una Raspberry Pi 4 que actua com a servidor local.
- Rep dades dels sensors via USB
- Distribueix la informació als usuaris mitjançant la Xarxa local.
- Permet accés remot seguir via VPN o configuracions de xarxa.

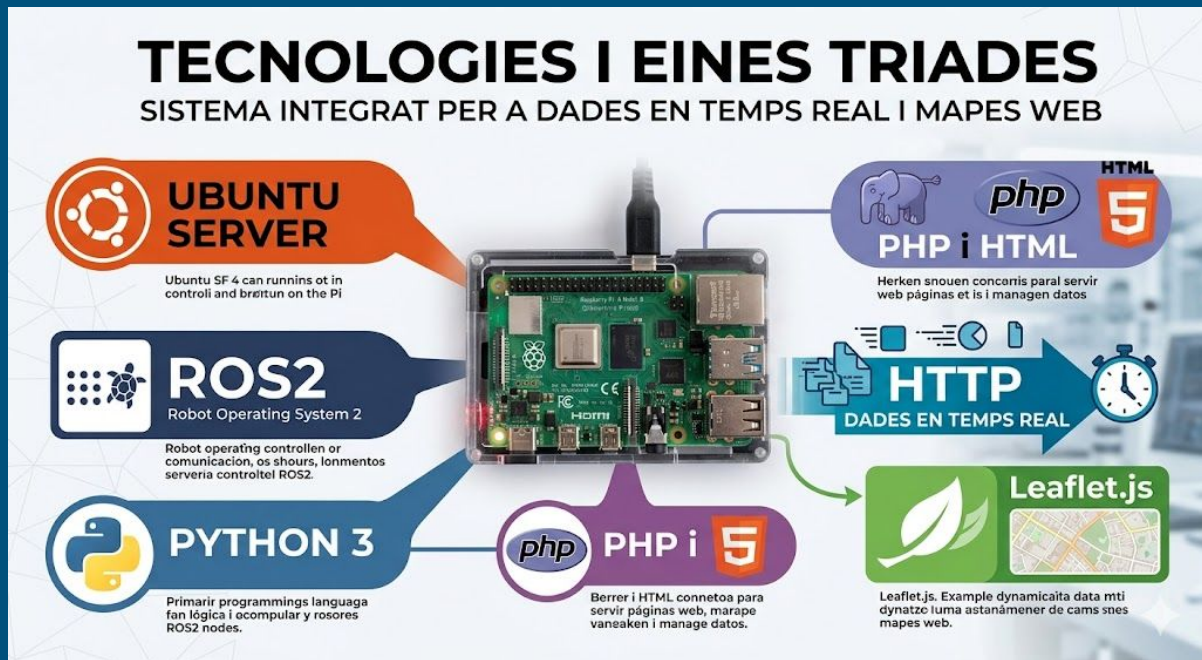


SOLUCIONS TÈCNIQUES



Tecnologies i eines triades:

- Raspberry Pi 4
- Ubuntu Server
- ROS2
- Python 3
- PHP i HTML
- HTTP dades en temps real
- Leaflet.js



PLANIFICACIÓ DEL PROJECTE



1 - Fase inicial

- Definició del sistema Navisafe
- Definició de les necessitats i límits que cobreix el sistema



PLANIFICACIÓ DEL PROJECTE



2 - Fase de planificació

- Definició de cada cas d'ús
- Descomposició en capes
- Creació del diagrama de Gantt



- Desenvolupament del panell de control a la web
- Implementació de ROS2
- Connexió amb sensors simulats
- Creació de base de dades
- Integració de mòduls

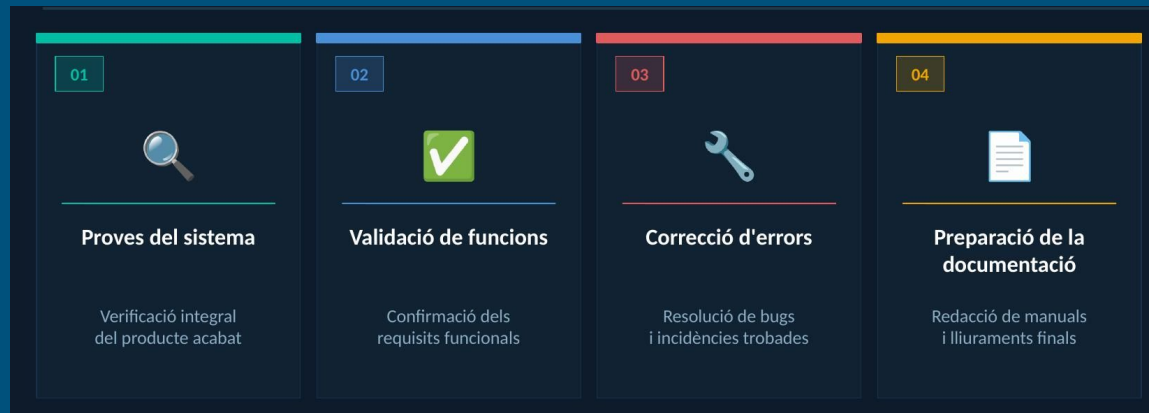


PLANIFICACIÓ DEL PROJECTE



4 - Fase de tancament

- Proves del sistema
- Validació de funcions
- Correcció d'errors
- Preparació de la documentació final

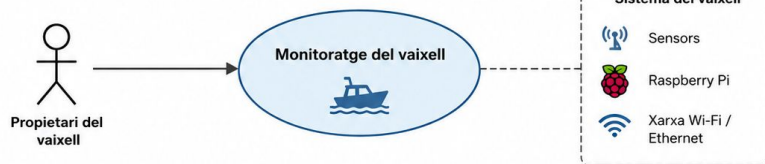


INTEGRACIÓ I ARQUITECTURA



FASE 1

CU-01: Monitoratge del vaixell



Funcionalitat: Supervisió remota de consumibles, motor i càmeres, amb alertes automàtiques i accés via navegador.



Precondicions:

- L'usuari ha iniciat sessió amb credencials vàlides.
- El sistema de sensors i Raspberry Pi està operatiu.
- La xarxa Wi-Fi / Ethernet del vaixell funciona.



Flux principal:

1



Accés al sistema.

2



Visualització de combustible, bateries i estat bomba centina.

3



Visualització de paràmetres del motor.

4



Visualització de càmeres embarcades.

5



Alertes automàtiques per incidències.



Fluxos alternatius:



Sensor fora de servei:

- si no es reben dades en 2 segons es considera timeout.
- A partir de 3 errors consecutius s'envia un avis i a 4 es dispara una alarma.
- Es pot notificar per correu i permetre consultar càmera o temperatura.



Sensor d'aigua a la sentina:

- si es detecta nivell alt d'aigua s'activa l'alarma immediatament.
- Es recomana fer un test abans de navegar o de manera setmanal per comprovar el correcte funcionament del sistema.



Connexió interrompuda:

- si es perd la connexió es fa un intent automàtic de reconexió i, si no es recupera, s'envia una alerta.



Postcondicions: Dades i alertes registrades amb traçabilitat.



Requisits especials:



Seguretat: Autenticació i permisos.



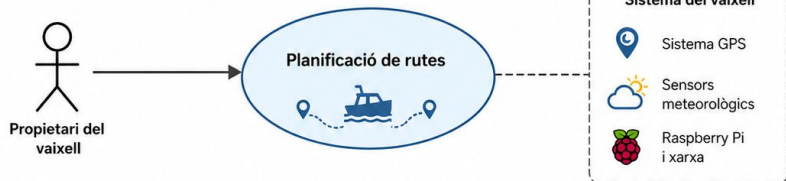
Integritat: Control de versions i logs d'auditoria.

INTEGRACIÓ I ARQUITECTURA



FASE 2

CU-02: Planificació de routes



Funcionalitat: Planificació de routes amb ajust automàtic segons condicions meteorològiques.



Precondicions:

- Sistema GPS i sensors meteorològics operatius.
- Sistema de control de routes en funcionament.
- Xarxa i Raspberry Pi actius.



Flux principal:

1



L'usuari prepara la ruta amb punts de parada i trajectes.

2



El sistema ajusta la ruta segons vent, onatge i corrents.

3



Es carrega la ruta al sistema del vaixell.

4



Es recalcula automàticament si canvien les condicions.

5



Visualitza routes i alertes a la interfície web.



Fluxos alternatius:



Predicció meteorològica no disponible
→ ruta mantinguda sense ajustos.



GPS fora de servei
→ recalcul automàtic bloquejat i alerta.



Routes fetes anteriorment i adaptarem a CDD
→ es poden reutilitzar i adaptar segons condicions actuals.



Postcondicions: Ruta optimitzada i segura.



Requisits especials:



Seguretat: Accés només a usuaris autoritzats.



Disponibilitat: Actualització de routes en temps real.



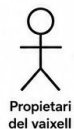
Integritat: Historial de routes amb versions i alertes documentades.

INTEGRACIÓ I ARQUITECTURA



FASE 4

CU-04: Revisió prèvia



Propietari
del vaixell



Revisió prèvia

Sistema del vaixell

- Raspberry Pi
- Sensors i dispositius
- Xarxa Wi-Fi / Ethernet
- Càmeres
- Components crítics (motor, bomba, timó, etc.)



Funcionalitat: Verificació del sistema abans de navegar mitjançant el navegador, assegurant que tots els sensors, dispositius i components crítics funcionen correctament.



Precondicions:

- L'usuari ha iniciat sessió amb credencials vàlides.
- El sistema Raspberry Pi està operatiu.
- Connexió de xarxa disponible (Wi-Fi o Ethernet).
- Sensors i dispositius connectats al sistema.



Flux principal:

- L'usuari accedeix al sistema des del navegador.
- Selecciona l'opció de "test abans de sortir".
- El sistema executa una comprovació automàtica dels sensors (combustible, bateries, temperatura, bomba de sentina, etc.).
- Es verifica el funcionament de la bomba de sentina mitjançant un test d'activació.
- Es comprova l'estat del motor i altres components crítics.
- Es validen les càmeres embarcades i la seva visualització.
- Es verifica el timó.
- El sistema mostra un informe amb l'estat de tots els elements (correcte / error).



Fluxos alternatius:

- Sensor no respon**
→ notificació després de timeout de 2 segons.
- 3 errors consecutius**
→ alerta al sistema.
- 4 errors consecutius**
→ enviament de notificació (correu electrònic o alerta remota).
- Error en la bomba de sentina**
→ alerta crítica de risc (possible acumulació d'aigua).
- Càmera no disponible**
→ notificació i opció de reintent.
- Connexió interrompuda**
→ intent de reconexió automàtic i alerta.



Postcondicions:

- Estat del sistema verificat abans de la navegació.
- Resultats del test registrats amb traçabilitat.
- Alertes guardades per revisió posterior.



Requisits especials:

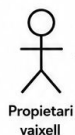
- Seguretat:** Accés restringit amb autenticació.
- Fiabilitat:** Execució completa del test sense interrupcions.
- Integritat:** Registre de logs i historial de comprovacions.
- Usabilitat:** Resultats clars i fàcilment interpretables per l'usuari.

INTEGRACIÓ I ARQUITECTURA



FASE 3

CU-03: Dades de la navegació



Propietari
vaixell

Dades de la navegació



Sistema del vaixell

- Sensors
- GPS
- Raspberry Pi
- Xarxa Wi-Fi / Ethernet



Funcionalitat: Selecció de paràmetres i generació de gràfics/informes.



Precondicions:

- Sistema de sensors operatiu.
- Dades disponibles: ruta GPS, velocitat i consum.
- Interfície web en funcionament.



Flux principal:

Visualització del pñanell del vaixell amb:

1



Estat del vaixell, consum
i combustible disponible



Navegació amb mapa
i carta nàutica



2



Visualització de dades en temps
real al navegador.

3



Genera gràfics i informes.

4



Accedeix a les dades tant a bord
com remotament.

5



Les dades es queden registrades
per anàlisi futura.



Fluxos alternatius:



Sensor no disponible
→ notificació d'error



Error en generació de gràfics
→ regeneració o exportació
de dades.



Postcondicions: Historial de dades disponible per anàlisi futura.



Requisits especials:

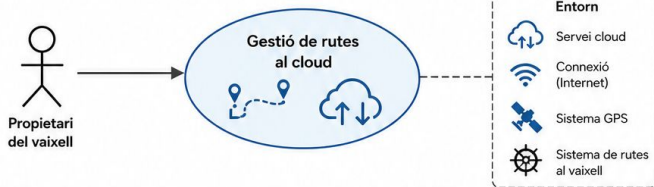
- **Rendiment:** Generació de gràfics sense retard perceptible.
- **Integritat:** Dades sense pèrdua ni corrupció.

INTEGRACIÓ I ARQUITECTURA



FASE 5

CU-05: Gestió de rutes al cloud



Funcionalitat: Pujada, emmagatzematge i descàrrega de rutes al cloud per a la seva reutilització i sincronització amb el vaixell.



Precondicions:

- L'usuari ha iniciat sessió amb credencials vàlides.
- Connexió activa
- Sistema de rutes i GPS operatiu
- Servei cloud disponible



Flux principal:

- 1 L'usuari crea o selecciona una ruta de navegació.
- 2 Puja la ruta al cloud des de la interfície web.
- 3 El sistema valida i guarda la ruta al servidor.
- 4 L'usuari pot accedir a les rutes guardades des de qualsevol dispositiu.
- 5 L'usuari selecciona una ruta del cloud.
- 6 La ruta es descarrega automàticament al sistema del vaixell.
- 7 La ruta queda disponible per a navegació directa o planificació.



Fluxos alternatius:



Error de connexió amb el cloud
→ reintent automàtic i notificació.



Ruta corrupta o incompleta
→ rebuig de pujada amb avis al usuari.



Conflicte de versions de ruta
→ el sistema manté historial i permet escollir versió.



Postcondicions:

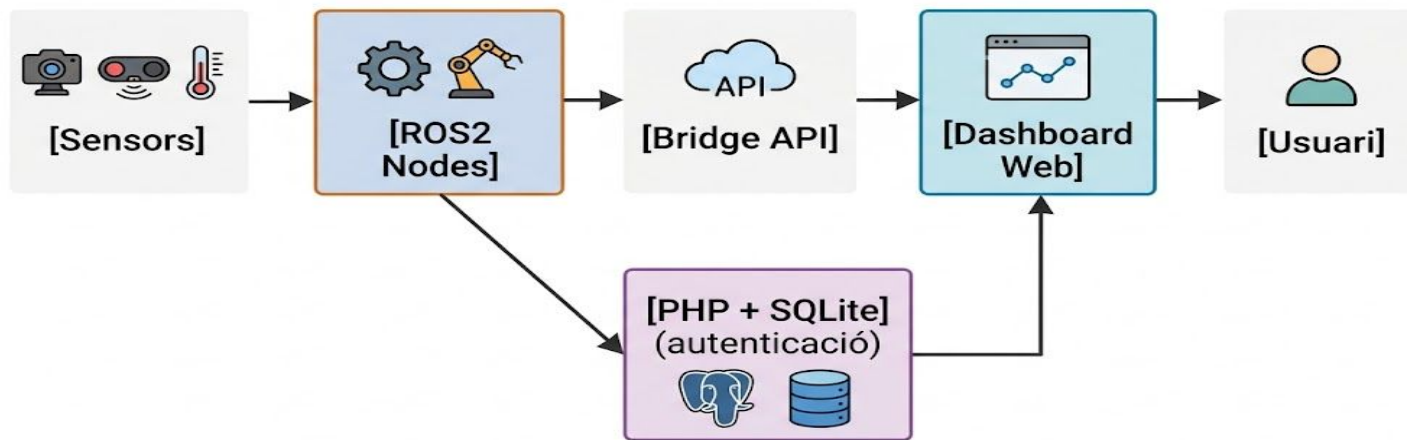
- Ruta emmagatzemada correctament al cloud.
- Ruta sincronitzada i disponible al vaixell.
- Historial de versions de rutes guardat.



Requisits especials:

- Seguretat:** Accés al cloud només amb usuari autenticat.
- Disponibilitat:** Sincronització de rutes en qualsevol moment.
- Integritat:** Dades de rutes sense pèrdua ni corrupció.
- Traçabilitat:** Registre de pujades, descàrregues i versions.

INTEGRACIÓ I ARQUITECTURA



OBTENCIÓ DE LES TASQUES



Cas d'ús / Capa	Capa Física	Xarxa i Comunicacions	Sistema Operatiu	Serveis	Aplicacions
CU-01 Monitoratge del vaixell	Connexió de sensors i actuadors a la Raspberry Pi	Configuració Wi-Fi i Ethernet	Instal·lació Ubuntu	ROS2 i servidor web	Dashboard i alertes
CU-02 Planificació de rutes	Configuració GPS i sensors	Accés remot i VPN	Configuració del sistema	Motor de càlcul de rutes	Mapa interactiu
CU-03 Dades de navegació	Sensors de consum, temperatura i actuadors	Comunicació segura HTTPS	Supervisió de serveis	Base de dades SQLite	Gràfics i informes
CU-04 Revisió prèvia	Test de sensors i actuadors	Verificació connexió	Control de processos	Servei de verificació	Panell de revisió
CU-05 Gestió de rutes cloud	Raspberry Pi i emmagatzematge	Connexió Internet i firewall	Gestió de sessions	APIs de rutes	Historial i sincronització

OBTENCIÓ DE LES TASQUES



T-01 Instal·lació de l'entorn base:

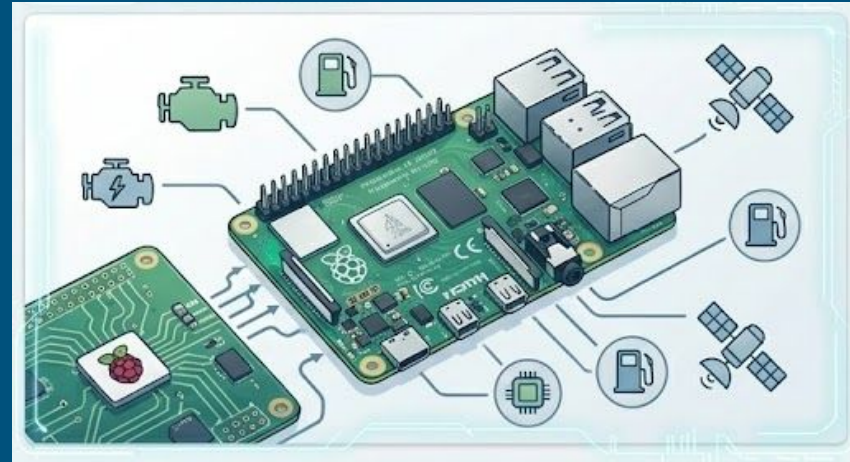
Preparació de la Raspberry Pi amb Ubuntu Server i configuració de la connexió remota.

Resultat:

Sistema Ubuntu funcional i accessible des de la xarxa.

DoD

- Ubuntu instal·lat
- SSH operatiu
- IP estàtica configurada



OBTENCIÓ DE LES TASQUES



T-02 Implementació dels nodes ROS2:

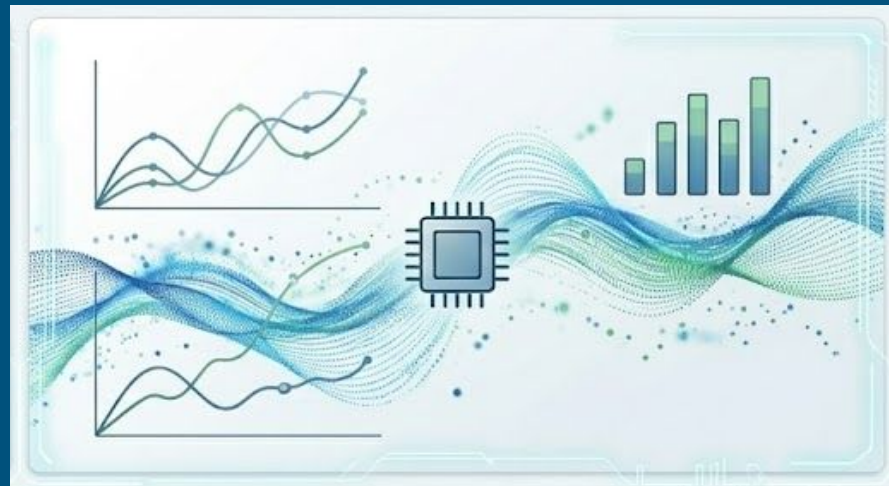
Creació dels nodes ROS2 per llegir les dades dels sensors.

Resultat:

Nodes GPS, Fuel, Motor i Sentina funcionals.

DoD

- Publicació de dades en temps real
- Detecció d'errors implementada
- Generació de fitxers JSON



OBTENCIÓ DE LES TASQUES



T-03 Backend PHP i base de dades:

Desenvolupament del sistema de login i emmagatzematge de rutes.

Resultat:

Sistema d'autenticació i historial de rutes funcional.

DoD

- Login amb sessions
- Base de dades SQLite
- APIs `save_route.php` i `get_routes.php`



OBTENCIÓ DE LES TASQUES



T-04 Landing page pública:

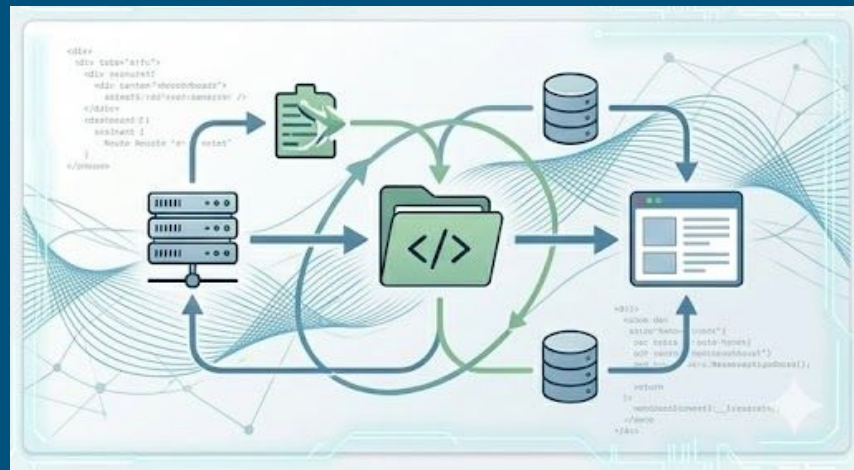
Creació de la pàgina principal informativa de Navisafe.

Resultat:

Web responsive amb informació del projecte.

DoD

- Disseny responsive
- Navegació funcional
- Accés al dashboard



OBTENCIÓ DE LES TASQUES



T-05 Validació i documentació final:

Realització de proves finals i preparació de la documentació.

Resultat:

Sistema completament provat i documentat.

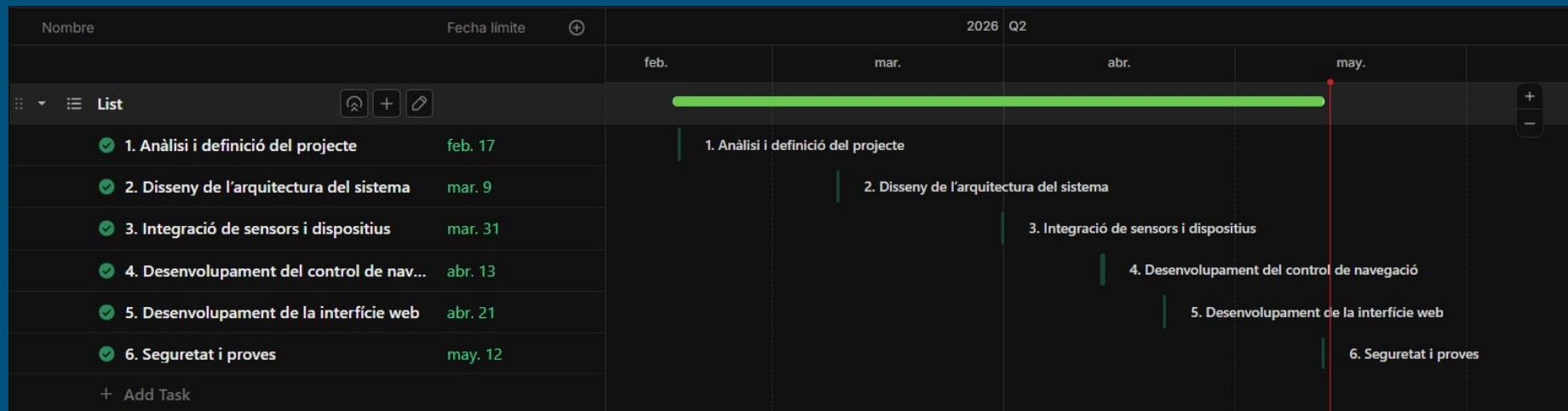
DoD

- Proves superades
- Dashboard funcional
- Documentació preparada

OBTENCIÓ DE LES TASQUES



DIAGRAMA DE GANT



WEB I PROGRAMA



El projecte inclou les següents pàgines principals:

Pàgina	Funció
index.html	Landing page pública
dashboard.html	Panell de control privat
Backend PHP (login.php, db.php)	Autenticació d'usuaris i gestió de sessions

WEB I PROGRAMA



Les dues pàgines es van crear amb HTML i CSS bàsic desenvolupat per nosaltres.

Després vam utilitzar IA per millorar:

- El disseny visual
- L'estructura de la web
- L'estil responsive
- La distribució dels elements

Fet manualment
Concepte i abast del projecte Navisafe
Estructura base d'<code>index.html</code> i <code>dashboard.html</code> HTML i CSS
Investigació d'Angler i conversió de C++ a Python Recerca i reescriptura dels nodes
Sistema d'autenticació (<code>login.php</code> / <code>db.php</code>) Implementació manual de sessions i validació d'usuaris
Arquitectura del sistema i casos d'ús Configuració de la Raspberry Pi i Ubuntu Server
Memòria tècnica i diagrama de Gantt

WEB I PROGRAMA



Ús de la intel·ligència artificial en el desenvolupament:

Fet amb IA
Millora del disseny visual d'index.html HTML/CSS de la landing page refinat amb IA a partir de la base pròpia
Millora del disseny visual de dashboard.html HTML/CSS del panell de control refinat amb IA a partir de la base pròpia
Ajuda per entendre el model de nodes ROS2 IA va ajudar a comprendre la sintaxi de Python
Arxiu bridge.py Codi del bridge entre ROS2 i el dashboard fet amb ajuda d'IA

DEMOSTRACIÓ DEL FUNCIONAMENT

[Funcionalitats](#)[Com funciona](#)[Preus](#)[Sobre](#)[Accedir →](#)

El teu vaixell, sempre **sota** **control**

Navisafe és la plataforma de monitoratge marítim que et permet supervisar tots els sensors del teu vaixell en temps real, des de qualsevol dispositiu.

[Accedir al sistema](#)[Descobrir més](#)

NAVISAFE

DEMOSTRACIÓ DEL FUNCIONAMENT



NAVI SAFE · Panell de control

● En directe



admin

Salir



Monitor



Maps



Alertes



Revisió



COMBUSTIBLE

58.9%

OK

MOTOR

111.2°C

CRITICAL



SENTINA

99.8%

CRITICAL



AIS

2 vaix.
COSTA BRAVA I...



CÀMERA

En
línia

OK



VELOCITAT

1.7 kn

Rumb: 318°

ALERTES RECIENTES

●	●	Sentina: 32.6%	23:09:41
●	●	Sentina: 31.3%	23:09:40
●	●	Sentina: 31%	23:09:38

RESUM SENSORS

SENSOR	VALOR	ESTAT
GPS	41.847771, 3.135037	OK
Fuel	75.5%	OK
Motor	99.1°C	WARNING
Sentina	32.6%	CRITICAL
AIS	0 vaix.	OK
Càmera	Offline	CRITICAL

EXEMPLE DEL CODI PHP



```
$db->exec("CREATE TABLE IF NOT EXISTS users (  
    id            INTEGER PRIMARY KEY AUTOINCREMENT,  
    username      TEXT NOT NULL UNIQUE,  
    password_hash TEXT NOT NULL  
)");  
  
$usuaris = [  
    ['admin', 'navisafe'],  
    ['guillem', '123'],  
    ['miquel', '123'],  
];  
  
foreach ($usuaris as [$nom, $contrasenya]) {  
    $check = $db->prepare("SELECT id FROM users WHERE username = ?");  
    $check->execute([$nom]);  
    if (!$check->fetch()) {  
        $hash = password_hash($contrasenya, PASSWORD_BCRYPT);  
        $ins = $db->prepare("INSERT INTO users (username, password_hash) VALUE  
        $ins->execute([$nom, $hash]);  
    }  
}  
  
return $db;
```


CONNEXIÓ ENTRE ROS2 I EL DASHBOARD



```
51 @app.route("/api/ais")
52 def ais(): return ok(read("ais_sensor.json"))
53
54 @app.route("/api/water")
55 def water(): return ok(read("water_sensor.json"))
56
57 @app.route("/api/camera")
58 def camera(): return ok(read("camera_node.json"))
59
60 @app.route("/api/alerts")
61 def alerts():
62     path = os.path.join(DATA_DIR, "alerts.json")
63     if not os.path.exists(path): return ok({})
64     try:
65         with open(path) as f: return ok(json.load(f)[-20:])
66     except: return ok({})
67
68 @app.route("/api/all")
69 def all_data():
70     alerts_data = []
71     path = os.path.join(DATA_DIR, "alerts.json")
72     if os.path.exists(path):
73         try:
74             with open(path) as f: alerts_data = json.load(f)[-10:]
75         except: pass
76     return ok({
77         "gps": read("gps_sensor.json"),
78         "fuel": read("fuel_sensor.json"),
79         "engine": read("engine_temp_sensor.json"),
80         "ais": read("ais_sensor.json"),
81         "water": read("water_sensor.json"),
82         "camera": read("camera_node.json"),
83         "alerts": alerts_data,
84         "timestamp": time.time(),
85         "time_str": time.strftime("%H:%M:%S"),
86     })
```

RESULTATS I VALIDACIÓ



Objectius aconseguits:

Funcionalitat	Estat	Observació
CU-01 Monitoratge en temps real	✓ Completat	Sensors simulats funcionant amb ROS2
CU-02 Planificació de rutes	✓ Completat	Mapa interactiu amb Leaflet.js
CU-03 Dades de navegació i gràfics	✓ Completat	Dashboard amb visualització en temps real
CU-04 Revisió prèvia al viatge	✓ Completat	Test automàtic de tots els sensors
CU-05 Gestió de rutes al cloud	✓ Completat	APIs PHP amb SQLite funcionals
Autenticació d'usuaris	✓ Completat	Login PHP amb sessions segures
Alertes automàtiques	✓ Completat	Detecció d'errors i notificacions actives
Landing page pública	✓ Completat	Web responsive i accessible

RESULTATS I VALIDACIÓ



Proves realitzades:

Component	Prova realitzada	Resultat	Observacions
Sensors simulats ROS2	Publicació de tòpics en temps real	✓ Completat	Dades actualitzades cada 2s
Bridge Python	Crida a /api/fuel, /api/motor	✓ Completat	Latència < 100ms
Panell de control HTML/JS	Visualització en Chrome i Firefox	✓ Completat	Responsive en mòbil
Login PHP	Accés amb credencials correctes/incorrectes	✓ Completat	Sessions gestionades correctament
Mapa Leaflet.js	Visualització de ruta GPS simulada	✓ Completat	Marcadors visibles
Alertes automàtiques	Simulació sensor fora de servei	✓ Completat	Alerta en 3 errors consecutius

RESULTATS I VALIDACIÓ



Feedback rebut:

El professor Francesc ha tingut un paper molt important durant el projecte. Ens ha ajudat a definir i organitzar la idea inicial, així com a orientar el guió de la presentació i el disseny de la interfície web. També va recomanar l'ús de ROS2 com a base del sistema, cosa que ha condicionat l'arquitectura general de Navisafe i ha permès treballar amb tecnologia utilitzada en robòtica real.

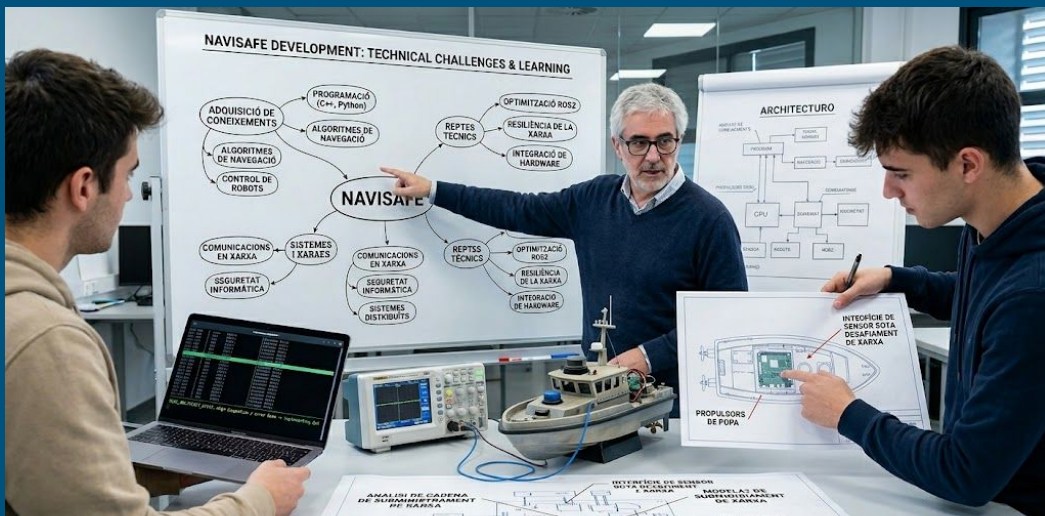


CONCLUSIONS I APRENENTATGES



Aprenentatges tècnics

El desenvolupament de Navisafe ha representat un repte tècnic, ja que ens ha permès adquirir i consolidar coneixements en àrees molt diverses de la informàtica i les xarxes.

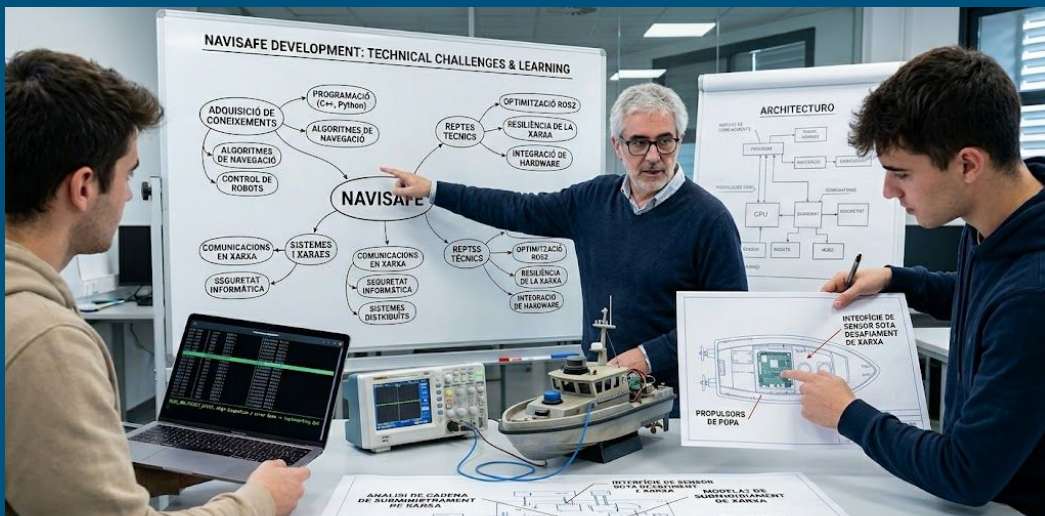


CONCLUSIONS I APRENENTATGES



Aprenentatges tècnics

El desenvolupament de Navisafe ha representat un repte tècnic, ja que ens ha permès adquirir i consolidar coneixements en àrees molt diverses de la informàtica i les xarxes.



CONCLUSIONS I APRENENTATGES



Un dels principals aprenentatges ha estat l'ús de **ROS2**, entenent el sistema de comunicació entre nodes amb publicadors i subscriptors. També hem aplicat **Python** de manera pràctica dins del projecte.

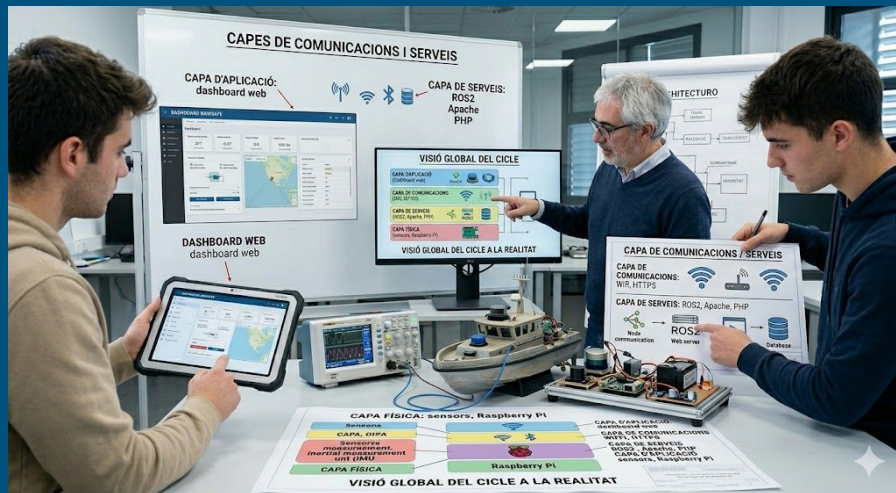


CONCLUSIONS I APRENENTATGES



Integració full-stack:

El projecte ens ha fet treballar en totes les capes d'un sistema informàtic: des de la capa física (sensors, Raspberry Pi) fins a la capa d'aplicació (dashboard web), passant per la capa de comunicacions (Wifi, HTTPS) i la capa de serveis (ROS2, Apache, PHP). Això ens ha donat una visió global del cicle a la realitat.





CONCLUSIONS I APRENENTATGES

Seguretat informàtica aplicada:

Ho hem implementat en l'autenticació amb sessions PHP, comunicació HTTPS amb certificat autosignat, control d'accés al dashboard i registre d'activitat per auditoria.

Aprenentatges en equip:

- Ús de Git i GitHub per a la gestió del codi i la col·laboració en equip
- Planificació i seguiment de tasques amb diagrames de Gantt
- Resolució conjunta de problemes tècnics complexos
- Documentació tècnica clara i estructurada per facilitar la comprensió del sistema

CONCLUSIONS I APRENENTATGES



Dificultats trobades:

Dificultat	Com s'ha resolt
Incompatibilitat de ROS2 C++ amb Raspberry Pi 4	Estudi del projecte Angler i conversió dels nodes a Python 3 seguint el seu model de publishers/subscribers
Comunicació entre ROS2 i el dashboard web	Creació del fitxer <code>bridge.py</code> com a capa intermèdia amb Flask API REST
Gestió de la concurrència a SQLite	Implementació del mode WAL i cues d'escriptura per evitar bloquejos
Disseny responsive del dashboard	Ús de CSS Grid i Media Queries per adaptar la interfície a mòbils i tauletes

**GRÀCIES PER LA
VOSTRE ATENCIÓ**



NAVISAFE